

AI Mental Support – All-in-One Project (Complete Code & Instructions)

Features: Chatbot conversation, Emotion detection, Stress prediction, Mood tracking dashboard, Resource recommendations, Optional voice (STT/TTS).

Safety Notice: This chatbot is for educational use and general well-being support. It is **not** a medical device. In any crisis or self-harm risk, contact local emergency services or a trusted professional immediately.

Directory Layout

```
ai_mental_support/
├── backend/
│   ├── app.py
│   ├── models.py
│   ├── config.py
│   ├── requirements.txt
│   └── ml/
│       ├── __init__.py
│       ├── emotion_model.py
│       └── stress_model.py
│   └── resources/
│       └── recommendations.json
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── database/
│   └── (auto-created SQLite file: mental_support.db)
└── README.md
```

Quick Start (How to Run)

Backend

```
cd ai_mental_support/backend
python -m venv venv
# Windows
venv\Scripts\activate
# macOS/Linux
# source venv/bin/activate
```

```
pip install -r requirements.txt
python app.py
```

Frontend

```
Option A: Double-click ai_mental_support/frontend/index.html to open in Chrome
Option B (recommended local server):
cd ai_mental_support/frontend
python -m http.server 5500
Visit http://127.0.0.1:5500/index.html
```

Backend Code

backend/requirements.txt

```
Flask==3.0.2
Flask-Cors==4.0.1
SQLAlchemy==2.0.32
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.5.1
joblib==1.4.2
```

backend/config.py

```
import os
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(os.path.dirname(BASE_DIR), 'database', 'mental_support.db')
SQLALCHEMY_DATABASE_URI = f'sqlite:/// {DB_PATH}'
SQLALCHEMY_ECHO = False
```

backend/models.py

```
from datetime import datetime
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, Text
from sqlalchemy.orm import declarative_base, relationship
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    username = Column(String(50), unique=True, nullable=False)
    messages = relationship('Message', back_populates='user')
    moods = relationship('MoodEntry', back_populates='user')
```

```
class Message(Base):
    __tablename__ = 'messages'
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'), nullable=False)
    sender = Column(String(10), nullable=False) # 'user' or 'bot'
    text = Column(Text, nullable=False)
    timestamp = Column(DateTime, default=datetime.utcnow)
    user = relationship('User', back_populates='messages')
```

```
class MoodEntry(Base):
    __tablename__ = 'mood_history'
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'), nullable=False)
    mood = Column(String(50), nullable=False)
    stress_level = Column(String(50), nullable=False)
    timestamp = Column(DateTime, default=datetime.utcnow)
    user = relationship('User', back_populates='moods')
```

```
# backend/ml/__init__.py
# (empty file to mark Python package)
```

backend/ml/emotion_model.py

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
# Tiny, synthetic starter dataset for demonstration.
```

```
TRAIN_DATA = [
    ("i feel so sad and tired", "sad"),
    ("i'm anxious about exams", "anxious"),
    ("i'm stressed with deadlines", "stressed"),
    ("i'm feeling happy today", "happy"),
    ("i feel okay and neutral", "neutral"),
    ("i can't stop worrying", "anxious"),
    ("i'm overwhelmed and stressed", "stressed"),
    ("i'm grateful and joyful", "happy"),
    ("i'm down and low", "sad"),
    ("just normal day", "neutral"),
]
```

```
class EmotionClassifier:
    def __init__(self):
        self.labels = ["sad", "anxious", "stressed", "happy", "neutral"]
        self.pipeline = Pipeline([
            ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=1)),
            ('clf', LogisticRegression(max_iter=1000))
        ])
```

```

def train(self):
    X = [t for t, _ in TRAIN_DATA]
    y = [l for _, l in TRAIN_DATA]
    self.pipeline.fit(X, y)
    return self

def predict(self, text: str) -> str:
    return self.pipeline.predict([text])[0]

```

backend/ml/stress_model.py

```

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

```

```

TRAIN_DATA = [
    ("i'm fine and relaxed", "Low"),
    ("i feel a bit worried", "Medium"),
    ("i'm extremely overwhelmed and panicking", "High"),
    ("manageable stress", "Medium"),
    ("calm and peaceful", "Low"),
    ("too many deadlines and can't sleep", "High"),
]

```

```

class StressClassifier:
    def __init__(self):
        self.labels = ["Low", "Medium", "High"]
        self.pipeline = Pipeline([
            ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=1)),
            ('clf', LogisticRegression(max_iter=1000))
        ])

    def train(self):
        X = [t for t, _ in TRAIN_DATA]
        y = [l for _, l in TRAIN_DATA]
        self.pipeline.fit(X, y)
        return self

    def predict(self, text: str) -> str:
        return self.pipeline.predict([text])[0]

```

backend/resources/recommendations.json

```

{
  "sad": {
    "tips": [
      "Write down three things that went okay today.",
      "Take a short walk and notice your surroundings.",
      "Reach out to a friend or family member."
    ],
    "content": [
      {"title": "Guided Loving-Kindness", "url": "https://www.youtube.com/results?search_query=loving+kindness+meditation"},
      {"title": "Understanding Low Mood", "url": "https://www.psychologytoday.com"}
    ]
  },
  "anxious": {
    "tips": [
      "Try 4-7-8 breathing for one minute.",
      "Limit caffeine and hydrate.",
      "Ground yourself: name 5 things you can see."
    ],
    "content": [
      {"title": "Box Breathing Tutorial", "url": "https://www.youtube.com/results?search_query=box+breathing"},
      {"title": "Anxiety Basics", "url": "https://www.anxiety.org"}
    ]
  },
  "stressed": {
    "tips": [
      "Break tasks into tiny steps and do one.",
      "Take a 5-minute stretch break.",
      "Schedule rest like a meeting."
    ],
    "content": [
      {"title": "10-Minute Body Scan", "url": "https://www.youtube.com/results?search_query=body+scan+meditation"},
      {"title": "Time Management Basics", "url": "https://www.mindtools.com"}
    ]
  },
  "happy": {
    "tips": [
      "Savor this moment: describe it in detail.",
      "Share your good news with someone.",
      "Do a small act of kindness."
    ],
  },
}

```

```

    "content": [
        {"title": "Gratitude Practice", "url": "https://www.youtube.com/results?search_query=gratitude+practice"}
    ],
    "neutral": {
        "tips": [
            "Do a 2-minute mindful check-in.",
            "Plan one enjoyable activity today.",
            "Drink water and stretch."
        ],
        "content": [
            {"title": "Mindful Check-in", "url": "https://www.youtube.com/results?search_query=mindful+check+in"}
        ]
    },
    "by_stress": {
        "Low": ["Maintain routines and sleep hygiene.", "Consider light exercise."],
        "Medium": ["Prioritize tasks with a quick to-do.", "Practice 5-minute breathing."],
        "High": ["Pause: breathe 4-7-8.", "Reduce commitments where possible, ask for help."]
    }
}

```

backend/app.py

```

from flask import Flask, request, jsonify
from flask_cors import CORS
from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session
import json, os
from datetime import datetime, timedelta

from config import SQLALCHEMY_DATABASE_URI
from models import Base, User, Message, MoodEntry
from ml.emotion_model import EmotionClassifier
from ml.stress_model import StressClassifier

app = Flask(__name__)
CORS(app)

# --- Database setup ---
engine = create_engine(SQLALCHEMY_DATABASE_URI, future=True)
Base.metadata.create_all(engine)

# Ensure a default demo user exists
with Session(engine) as session:
    user = session.query(User).filter_by(username='demo').first()
    if not user:
        user = User(username='demo')
        session.add(user)
        session.commit()

# --- ML models (trained at startup on small synthetic data) ---
emotion_model = EmotionClassifier().train()
stress_model = StressClassifier().train()

# --- Load resource recommendations ---
RESOURCES_PATH = os.path.join(os.path.dirname(__file__), 'resources', 'recommendations.json')
with open(RESOURCES_PATH, 'r', encoding='utf-8') as f:
    RECS = json.load(f)

# --- Helper: simple empathetic reply generator ---
EMPATHY_OPENERS = {
    'sad': "I'm sorry you're going through this.",
    'anxious': "That sounds really tense and worrying.",
    'stressed': "It makes sense you're feeling stretched.",
    'happy': "Love to hear that!",
    'neutral': "Thanks for sharing."
}

def make_reply(msg: str, emotion: str, stress: str) -> str:
    opener = EMPATHY_OPENERS.get(emotion, "I hear you.")
    tip = RECS.get(emotion, {}).get('tips', ["Let's take it one step at a time."])[0]
    if stress == 'High' and emotion in ['sad', 'anxious', 'stressed']:
        crisis_line = ("If you ever feel unsafe or at risk of self-harm, please contact local emergency services")
    else:
        crisis_line = ""
    return f"{opener} {tip}{crisis_line}"

@app.get('/api/ping')
def ping():
    return {"ok": True, "message": "AI Mental Support Backend Running"}

@app.post('/api/chat')
def chat():

```

```

data = request.get_json(force=True)
text = (data or {}).get('message', '').strip()
username = (data or {}).get('username', 'demo')
if not text:
    return jsonify({"error": "message is required"}), 400

with Session(engine) as session:
    user = session.query(User).filter_by(username=username).first()
    if not user:
        user = User(username=username)
        session.add(user)
        session.commit()

    # Save user message
    umsg = Message(user_id=user.id, sender='user', text=text)
    session.add(umsg)

    # Predict emotion & stress
    emotion = emotion_model.predict(text)
    stress = stress_model.predict(text)

    # Save mood entry
    entry = MoodEntry(user_id=user.id, mood=emotion, stress_level=stress)
    session.add(entry)

    # Compose bot reply
    reply_text = make_reply(text, emotion, stress)
    bmsg = Message(user_id=user.id, sender='bot', text=reply_text)
    session.add(bmsg)
    session.commit()

    return jsonify({
        "reply": reply_text,
        "emotion": emotion,
        "stress": stress,
        "timestamp": entry.timestamp.isoformat()
    })

@app.get('/api/moods')
def moods():
    username = request.args.get('username', 'demo')
    days = int(request.args.get('days', 30))
    since = datetime.utcnow() - timedelta(days=days)
    with Session(engine) as session:
        user = session.query(User).filter_by(username=username).first()
        if not user:
            return jsonify({"items": []})
        q = session.query(MoodEntry).filter(MoodEntry.user_id==user.id, MoodEntry.timestamp>=since).order_by(MoodEntry.timestamp)
        items = [{
            'mood': m.mood,
            'stress': m.stress_level,
            'timestamp': m.timestamp.isoformat()
        } for m in q]
        return jsonify({"items": items})

@app.get('/api/resources')
def resources():
    emotion = request.args.get('emotion', 'neutral')
    stress = request.args.get('stress', 'Low')
    base = RECS.get(emotion, RECS['neutral'])
    stress_tips = RECS.get('by_stress', {}).get(stress, [])
    return jsonify({
        'emotion': emotion,
        'stress': stress,
        'tips': base.get('tips', []) + stress_tips,
        'content': base.get('content', [])
    })

if __name__ == '__main__':
    app.run(debug=True)

```

README (for reference)

AI Mental Support - All-in-One Starter

Features: chatbot conversation, emotion detection, stress prediction, mood tracking (chart), resource suggestions

Run Backend

cd backend

python -m venv venv

Windows: venv\Scripts\activate

macOS/Linux: source venv/bin/activate

pip install -r requirements.txt

python app.py

Run Frontend

Option A: open `frontend/index.html` in Chrome

Option B (recommended):

cd frontend

python -m http.server 5500

Open <http://127.0.0.1:5500/index.html>

Notes

- ML models use small synthetic datasets; replace with real datasets for accuracy.
- Data stored in `database/mental\_support.db` (SQLite).
- Educational use only; not medical advice.

Switching to MySQL (Optional)

1) Install driver: `pip install pymysql`

2) Create database: `CREATE DATABASE mental_support;`

3) Edit backend/config.py:

`SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://<user>:<password>@127.0.0.1:3306/mental_support'`

4) Run backend again so tables auto-create.